

Лабораторная работа № 12

Операционные системы — НПИбд-01-25

Взваров Михаил

В отчёте представлены результаты выполнения лабораторной работы № 12 по дисциплине «Операционные системы».

Приведены ход выполнения работы, полученные результаты и выводы.

Содержание

1	Цель работы	6
2	Задание	7
3	Выполнение лабораторной работы	8
3.1	1. Создание резервной копии командного файла	8
3.2	2. Обработка произвольного количества аргументов	9
3.3	3. Реализация аналога команды ls	11
3.4	4. Подсчёт файлов заданного типа	12
4	Вывод	14
5	Контрольные вопросы	15
5.1	1. Что такое командный процессор? Приведите примеры. Чем они отличаются?	15
5.2	2. Что такое POSIX?	15
5.3	3. Как определяются переменные и массивы в Bash?	16
5.4	4. Для чего предназначены команды let и read?	16
5.5	5. Какие арифметические операции можно выполнять в Bash? . .	17
5.6	6. Что означает конструкция (())?	17
5.7	7. Какие стандартные имена переменных используются в Bash? .	17
5.8	8. Что такое метасимволы?	18
5.9	9. Как экранировать метасимволы?	18
5.10	10. Как создать и запустить командный файл?	18
5.11	11. Как определяются функции в Bash?	19
5.12	12. Как определить, является объект каталогом или обычным фай- лом?	19
5.13	13. Для чего предназначены команды set, typeset и unset?	20
5.14	14. Как передаются параметры в командный файл?	20
5.15	15. Какие специальные переменные Bash используются и для чего? .	21
6	Список литературы	22

Список иллюстраций

3.1	Листинг командного файла backup_script.sh	8
3.2	Результат выполнения backup_script.sh	9
3.3	Листинг командного файла args.sh	10
3.4	Результат запуска args.sh с двенадцатью аргументами	10
3.5	Листинг командного файла myls.sh	11
3.6	Результат выполнения myls.sh	12
3.7	Листинг командного файла count_files.sh	13
3.8	Результат выполнения count_files.sh	13

Список таблиц

1 Цель работы

Изучить основы программирования в командном процессоре ОС UNIX/Linux и приобрести практические навыки написания небольших командных файлов на языке командного процессора Bash.

2 Задание

В ходе лабораторной работы необходимо:

1. Написать командный файл, создающий резервную копию самого себя в каталоге ~/backup.
2. Написать командный файл, обрабатывающий произвольное количество аргументов командной строки, в том числе более десяти.
3. Написать аналог команды ls, не используя команды ls и dir, с выводом информации о содержимом каталога и правах доступа.
4. Написать командный файл, принимающий расширение файла и путь к каталогу и подсчитывающий количество файлов указанного типа.

3 Выполнение лабораторной работы

3.1 1. Создание резервной копии командного файла

Для выполнения первого задания был использован командный файл `backup_script.sh`. Скрипт создаёт каталог для резервных копий и архивирует собственный файл.

```
1 #!/bin/bash
2 if [ ! -d ~/backup ]; then
3     mkdir -p ~/backup
4     echo "Каталог ~/backup создан."
5 fi
6 SCRIPT_NAME=$(basename "$0")
7 BACKUP_NAME="backup_$(date +%Y%m%d_%H%M%S).tar.gz"
8
9 tar -czf ~/backup/"$BACKUP_NAME" "$0"
10
11 echo "Резервная копия создана: ~/backup/$BACKUP_NAME"
michailvzarov@fedora:~/reports/lab12$
```

Рисунок 3.1: Листинг командного файла `backup_script.sh`

После запуска `backup_script.sh` была создана резервная копия командного файла в каталоге `~/backup`.

```
Резервная копия создана: ~/backup/~BACKUP_NAME
=== BACKUP FILES ===
/home/michailvzvarov/backup/backup_20260804_202925.tar.gz
/home/michailvzvarov/backup/backup_20260804_211951.tar.gz
/home/michailvzvarov/backup/backup_20260804_212743.tar.gz
michailvzvarov@fedora:~/reports/lab12$
```

Рисунок 3.2: Результат выполнения backup_script.sh

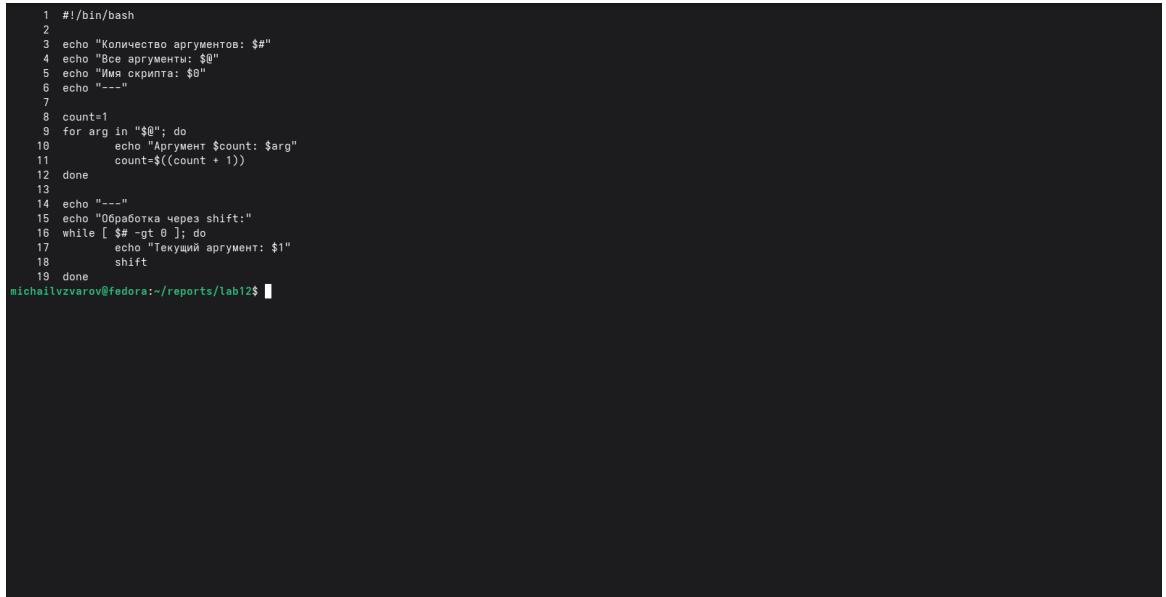
Таким образом, командный файл корректно создаёт архивную резервную копию самого себя.

3.2 2. Обработка произвольного количества аргументов

Во втором задании использован командный файл args.sh. Для последовательной обработки параметров командной строки применяется цикл и сдвиг позиционных параметров.

```

1  #!/bin/bash
2
3  echo "Количество аргументов: $#"
```



```

4  echo "Все аргументы: $@"
5  echo "Имя скрипта: $0"
6  echo "---"
7
8  count=1
9  for arg in "$@"; do
10     echo "Аргумент $count: $arg"
11     count=$((count + 1))
12 done
13
14 echo "---"
15 echo "Обработка через shift:"
16 while [ $# -gt 0 ]; do
17     echo "Текущий аргумент: $1"
18     shift
19 done
michailvzarov@fedora:~/reports/lab12$
```

Рисунок 3.3: Листинг командного файла args.sh

Для проверки работы программы ей было передано двенадцать аргументов. Это позволяет проверить корректную обработку количества параметров, превышающего десять.

```

Количество аргументов: 12
Все аргументы: one two three four five six seven eight nine ten eleven twelve
Имя скрипта: ./args.sh
---
./args.sh: строка 9: синтаксическая ошибка рядом с неожиданным маркером «do»
./args.sh: строка 9: `for arg in "$@"; do`
michailvzarov@fedora:~/reports/lab12$
```

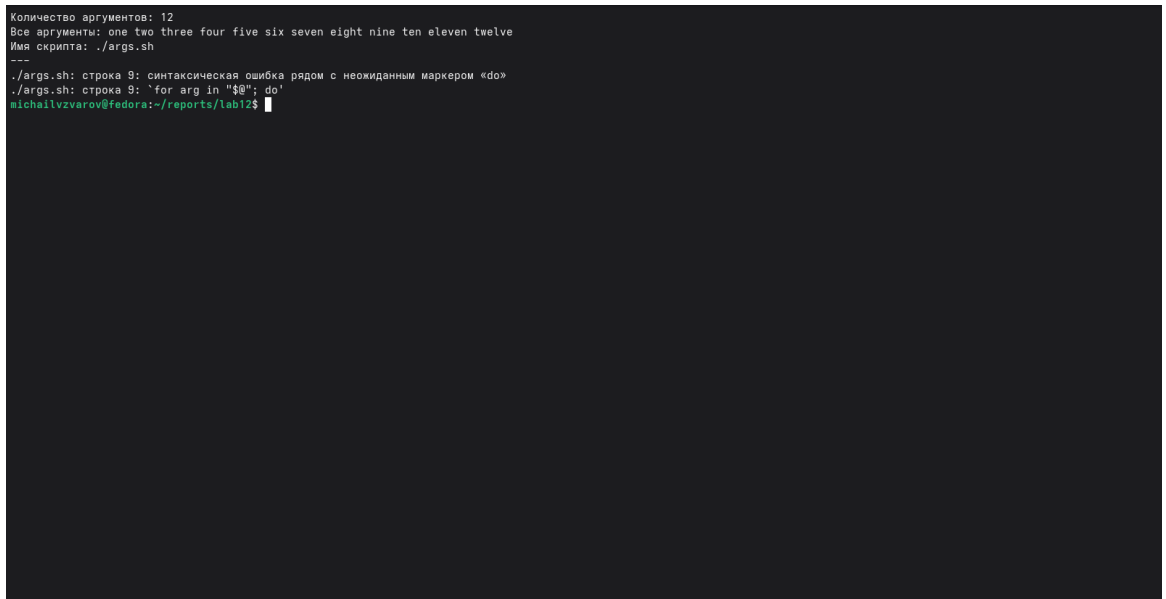


Рисунок 3.4: Результат запуска args.sh с двенадцатью аргументами

В результате все переданные аргументы были последовательно обработаны

и выведены на экран.

3.3 3. Реализация аналога команды ls

В третьем задании был создан командный файл `myls.sh`. Скрипт просматривает содержимое указанного каталога без использования команд `ls` и `dir`, определяет тип объектов и доступность операций чтения, записи и выполнения.

```
1 #!/bin/bash
2 DIR="${1:-.}"
3
4 if [ ! -d "$DIR"; then
5     echo "Ошибка: $DIR не является каталогом"
6     exit 1
7 fi
8
9 echo "Содержимое каталога: $DIR"
10 echo "=====
11
12 for file in "$DIR"/*; do
13     if [ -e "$file" ]; then
14         perms=""
15         [ -r "$file" ] && perms="${perms}r" || perms="${perms}-"
16         [ -w "$file" ] && perms="${perms}w" || perms="${perms}-"
17         [ -x "$file" ] && perms="${perms}x" || perms="${perms}-"
18
19         if [ -d "$file" ]; then
20             type="[DIR]"
21         elif [ -f "$file" ]; then
22             type="[FILE]"
23         elif [ -L "$file" ]; then
24             type="[LINK]"
25         else
26             type="[OTHER]"
27         fi
28
29         basename=$(basename "$file")
30         echo "$type $perms $basename"
31     fi
32 done
```

michailvzarov@fedora:~/reports/lab12\$

Рисунок 3.5: Листинг командного файла `myls.sh`

После запуска скрипта для каталога лабораторной работы была выведена информация о находящихся в нём объектах и доступных правах.

```
./mys.sh: строка 4: [: отсутствует символ «]»  
Содержимое каталога: /home/michailvzarov/reports/lab12  
=====  
[FILE] rwx args.sh  
[FILE] rwx backup_script.sh  
[FILE] rwx count_files.sh  
[DIR] rwx images  
[FILE] rwx myls.sh  
michailvzarov@fedora:~/reports/lab12$
```

Рисунок 3.6: Результат выполнения myls.sh

Полученный результат показывает, что написанный командный файл выполняет требуемый просмотр содержимого каталога и анализ прав доступа.

3.4 4. Подсчёт файлов заданного типа

В четвёртом задании использован командный файл `count_files.sh`. Скрипт получает два параметра: расширение файлов и путь к каталогу. После этого выполняется подсчёт обычных файлов соответствующего типа.

```

1  #!/bin/bash
2
3  if [ $# -ne 2 ]; then
4      echo "Использование: $0 <расширение> <директория>"
5      echo "Пример: $0 .txt /home/user"
6      exit 1
7  fi
8
9  EXT="$1"
10 DIR="$2"
11
12 if [ ! -d "$DIR" ]; then
13     echo "Ошибка: Директория $DIR не существует"
14     exit 1
15 fi
16
17 count=$(find "$DIR" -type f -name "$EXT" 2>/dev/null | wc -l)
18
19 echo "Количество файлов с расширением '$EXT' в директории '$DIR': $count"
michailvzarov@fedora:~/reports/lab12$

```

Рисунок 3.7: Листинг командного файла count_files.sh

Работа скрипта была проверена на каталоге лабораторной работы.

```

Количество файлов с расширением '.sh' в директории '/home/michailvzarov/reports/lab12': 4
michailvzarov@fedora:~/reports/lab12$

```

Рисунок 3.8: Результат выполнения count_files.sh

Результат подтверждает, что программа корректно принимает параметры командной строки и подсчитывает файлы с заданным расширением.

4 Вывод

В ходе лабораторной работы были изучены основные возможности программирования в командном процессоре Bash. Были рассмотрены позиционные параметры, циклы, условные конструкции, проверка типов файлов и прав доступа, а также работа с каталогами и архивами. Практически реализованы четыре командных файла: для резервного копирования, обработки произвольного количества аргументов, просмотра содержимого каталога и подсчёта файлов заданного типа. Полученные результаты подтвердили корректность работы созданных сценариев.

5 Контрольные вопросы

5.1 1. Что такое командный процессор? Приведите примеры. Чем они отличаются?

Командный процессор, или shell, — это программа, обеспечивающая взаимодействие пользователя с операционной системой посредством команд. Он принимает команды пользователя, интерпретирует их и организует выполнение соответствующих программ. Примерами являются sh, csh, ksh и bash. Они отличаются синтаксисом, набором встроенных команд, средствами программирования и дополнительными возможностями интерактивной работы.

5.2 2. Что такое POSIX?

POSIX (Portable Operating System Interface) — это семейство стандартов, определяющих унифицированный интерфейс операционных систем UNIX-подобного типа. Стандартизация позволяет создавать программы и командные файлы, которые легче переносить между различными совместимыми системами.

5.3 3. Как определяются переменные и массивы в Bash?

Переменная задаётся присваиванием без пробелов около знака равенства:

```
name=value
```

Обращение к значению выполняется с помощью символа \$:

```
echo $name
```

Массив можно определить, например, следующим образом:

```
array=(one two three)
```

Для обращения к элементу массива используется конструкция:

```
${array[0]}
```

5.4 4. Для чего предназначены команды let и read?

Команда `let` используется для выполнения целочисленных арифметических операций и изменения значений переменных. Команда `read` считывает данные из стандартного ввода и сохраняет их в переменных командного процессора.

Пример:

```
read number
```

```
let number=number+1
```

5.5 5. Какие арифметические операции можно выполнять в Bash?

Bash поддерживает сложение, вычитание, умножение, деление, получение остатка от деления, возведение в степень, инкремент и декремент, а также операции сравнения и присваивания.

Например:

```
result=$((a + b))
```

```
result=$((a * b))
```

```
result=$((a % b))
```

5.6 6. Что означает конструкция (())?

Конструкция (()) используется для вычисления арифметических выражений в Bash. Внутри неё можно выполнять вычисления, изменять переменные и использовать арифметические сравнения.

Например:

```
((count++))
```

```
((result = a + b))
```

5.7 7. Какие стандартные имена переменных используются в Bash?

В командном процессоре используются различные стандартные переменные окружения и shell-переменные. Например, HOME содержит путь к домашнему каталогу пользователя, PATH — список каталогов поиска исполняемых

программ, USER — имя пользователя, PWD — текущий рабочий каталог, SHELL — используемый командный процессор.

5.8 8. Что такое метасимволы?

Метасимволы — это символы, имеющие специальное значение для командного процессора. К ним относятся, например, *, ?, \$, &, |, ;, <, >, круглые и квадратные скобки. Они применяются для подстановок, перенаправления потоков, построения конвейеров и управления выполнением команд.

5.9 9. Как экранировать метасимволы?

Для отмены специального значения отдельного символа используется обратная косая черта \. Также применяются одинарные и двойные кавычки. Одинарные кавычки подавляют интерпретацию большинства специальных символов, а двойные позволяют выполнять некоторые подстановки, например подстановку переменных.

5.10 10. Как создать и запустить командный файл?

Необходимо создать текстовый файл, записать в него команды и обычно указать интерпретатор в первой строке:

```
#!/bin/bash
```

После этого файлу можно предоставить право на выполнение:

```
chmod +x script.sh
```

И запустить:

```
./script.sh
```

Также командный файл можно передать интерпретатору непосредственно:

```
bash script.sh
```

5.11 11. Как определяются функции в Bash?

Функцию можно определить следующим образом:

```
function hello {  
    echo "Hello"  
}
```

или:

```
hello() {  
    echo "Hello"  
}
```

После определения функция вызывается по своему имени:

```
hello
```

5.12 12. Как определить, является объект каталогом или обычным файлом?

Для проверки типа объекта используются условные выражения. Ключ `-d` проверяет, является ли объект каталогом, а `-f` — обычным файлом:

```
if [ -d "$name" ]; then
    echo "directory"
fi
```

```
if [ -f "$name" ]; then
    echo "regular file"
fi
```

5.13 13. Для чего предназначены команды set, typeset и unset?

Команда `set` используется для управления параметрами и режимами работы командного процессора, а также для установки позиционных параметров. `typeset` позволяет объявлять переменные и задавать их свойства. `unset` удаляет ранее определённую переменную или функцию.

5.14 14. Как передаются параметры в командный файл?

Параметры указываются после имени командного файла при его запуске:

```
./script.sh first second third
```

Внутри сценария к ним можно обращаться через позиционные параметры `$1`, `$2`, `$3` и т. д. `$0` содержит имя самого командного файла. Для обработки большого количества аргументов удобно использовать `$@`, `$#` и команду `shift`.

5.15 15. Какие специальные переменные Bash используются и для чего?

К основным специальным параметрам относятся:

- `$0` — имя выполняемого командного файла;
- `$1, $2, ...` — позиционные параметры;
- `$#` — количество переданных параметров;
- `$*` — все позиционные параметры;
- `$@` — все позиционные параметры с сохранением их разделения;
- `$?` — код завершения последней команды;
- `$$` — идентификатор текущего процесса shell;
- `$_` — идентификатор последнего процесса, запущенного в фоновом режиме.

6 Список литературы

1. Методические материалы к лабораторной работе «Программирование в командном процессоре ОС UNIX. Командные файлы».
2. Документация командного процессора Bash (`man bash`).